One node entry point, two protocol cores

Class-and-call-flow layout inspired by the supplied Wahoo++ figure. Solid arrows show ownership/calls; dashed arrows show shared service use.

node/ + core-api::ProtocolFactory

`node/src/shared_main.rs -> node/src/lib.rs -> core-api/src/lib.rs:44`

`--protocol delugi | raptr + opaque protocol JSON configuration`

`name() | run(configuration: &[u8], id: u32) | generate_keys() | verify_archive() | verify_ledger()`

core/delugi/

`factory.rs -> process.rs -> composed::Node`

`ORDERING replica / slot / history / scanner`

HISTORY3, CLOSED, catch-up, direct commit and ordered delivery

`AVAILABILITY availability / batch / mmr`

REPORT/OBS, authenticated bases, batch custody, FETCH/DATA

`STORAGE + OUTPUT record_store / composed / workload`

One synchronous actor loop; exact batch expansion and FIFO output

core/raptr/

`node/factory.rs + native/ + qs/ + transport/`

`ORDERING native/src/raptr + framework`

Pinned Raptr state machine, certificates and native timer policy

`AVAILABILITY qs/ + node/dissemination.rs`

Native Quorum Store batching, custody, proofs and payload repair

`OUTPUT src/output.rs + node/output.rs`

Tokio actors; reserve commit order before asynchronous retrieval

shared/ mechanisms used by the cores

`network + network-async`	`mempool + output`	`crypto + crypto-aptos`	`store + store-rocksdb`	`runtime + runtime-async`	`metrics`
Bounded TCP; async facade	Timed ingress; FIFO queues	Core-specific proof formats	Visible / Written barriers	Distinct execution profiles	Traffic + operation counters

Code boundary: core-api/src/lib.rs:44; node/src/shared_main.rs:4; core/delugi/src/process.rs:1983; core/raptr/node/src/factory.rs:128.
No universal consensus-transition trait exists today. OperationId is declared but is not wired into runtime calls. Detailed source index accompanies this PDF.

Delugi: composition drives an event/effect core

Public entry signatures are shortened only by omitting &mut self and common Result wrappers. Clock values are the core logical millisecond time.

produce

receive

time

composed::Node

core/delugi/src/composed.rs:144,210,242,385

state: ordering: Replica; store: BatchStore

pending_ordered; batch_ids; transaction_ids

produce(now, payload: Vec<u8>)

receive(now, from: Id, Message)

receive_ordering(now, wire::Message)

start(now); tick(now)

REPORT/OBS -> all-current (value x, witness pi). All entry calls return Result<Vec<composed::Effect>, String>.

(x, pi)

Q(r,x,pi)

effects

replica::Replica

core/delugi/src/replica.rs:2177,2384-2430

Inputs = BTreeMap<round, (value, witness)>

Q: Fn(round, &[u8], &[u8]) -> bool

start(now); receive(now, Message)

tick(now)

supply_input(now, round, value, witness)

-> Result<Vec<replica::Effect>, String>

slot / history / scanner keep voting, catch-up and delivery policy inside the core.

accept_ordering

composed effects

Node materialization (same state owner)

core/delugi/src/composed.rs:532-671

1 Decode descriptor; enqueue in pending_ordered.

2 Missing exact suffix -> FETCH; hold FIFO head.

3 After verified DATA: expand suffix; exact-batch dedup.

4 Decode requests; append contiguous application output.

process::Runtime::effects

core/delugi/src/process.rs:959; input Vec<composed::Effect>

Send {to,message} -> encode / bundle / TCP

Wake {at} -> shared TimerQueue

DirectCommit / Deliver -> observation log

Notice -> diagnostic events

Key separation: DirectCommit != ordered descriptor delivery != payload-verified application output.
Q checks immutable availability evidence; ordering does not wait for application materialization. TCP ACKs retire bounded in-flight frames, not each next send.

Raptr: native handlers plus a dissemination contract

The native Protocol and DisseminationLayer APIs stay in core/raptr; Context connects them to injected service adapters. Handler calls are async.

native::framework::Protocol

core/raptr/native/src/framework/mod.rs:36-134

```
associated Message; TimerEvent
start_handler(ctx)
message_handler(ctx, from, message)
module_event_handler(ctx, module, event)
timer_event_handler(ctx, event)
condition_handler(ctx)
```

run_ctx holds native state ownership for handler plus condition evaluation.

pull

prefix

Native DisseminationLayer

native/src/raptr/dissemination/mod.rs:81; node/dissemination.rs:204

```
prepare_block(round, excludes, authors)
-> Future<Payload>
available_prefix(payload, cached)
-> Future<(Prefix, BitVec)>
check_payload(payload) -> Result<(), BitVec>
notify_commit_identified(payloads, ts, voters)
```

CommittedPayload binds block_digest, payload and include_proofs.

Context + shared service adapters

framework/context.rs:19; node/factory.rs:128

```
send / unicast / multicast; notify(module,event)
set_timer(duration,event); next_event(); halt()
```

NativeDissemination uses real QS actors for pool pulls, certified batches, storage and RPC repair.

Executor: Tokio 2 workers / max 32 blocking threads; Rayon 2. Native timing semantics are retained.

Commit reservation -> Materializer -> output

node/dissemination.rs:262; node/output.rs:116

1 reserve exact commit order before asynchronous fetch.

2 Native QS retrieval + actual payload verification.

3 resolve completed sections; pop_ready only at FIFO head.

4 Workload publisher appends verified contiguous output.

Expansion: proof batches first, then original subblock order. Raw duplicates remain; useful credit is deduplicated.

Common mechanics: shared_mempool::Worker::start; shared_output::ResolutionQueue::{reserve,resolve,pop_ready}; shared_network::Transport.
Protocol messages, certificate validity, native batch cutting and output expansion remain core-owned. Raptr uses BCS/BLS/SHA3; Delugi uses its own signed wire format.

Workload, batch, payload and delay controls

Defaults below belong to the current runner or native configuration, not a proposed matched experiment. CLI = runner flag; JSON = node config; FIXED = source/profile.

PARAMETER	DELUGI TODAY	RAPTR TODAY	EXPOSURE / EVALUATION MEANING
Transaction size	CLI --useful-bytes=472. Canonical encoded request = useful + 60 bytes (532 at default).	CLI --useful-bytes=128. Native signed BCS size is measured; accepted 128-byte payload produced 409-byte transactions.	Match useful bytes; also record actual serialized sizes. Equal useful size does not mean equal wire size.
Input rate / horizon	CLI --rate=1000 total tx/s; warmup 10 s, measure 30 s, drain 30 s. Independent open-loop ingress.	CLI --interval-us=1000; --counts=16/origin. Four origins imply nominal 4000 tx/s, but only 64 finite requests.	Raptr runner is finite conformance, not a sustained-load adapter yet. Preserve rejection and unfinished counts.
Batch tx count	CLI --max-batch-transactions=128; also constrained by encoded byte capacity.	JSON quorum_store.sender_max_batch_txns=500; sender_max_total_txns=500; sender_max_num_batches=1 per pool pull.	Count limits are available. Batch cutting remains core policy; settings alone do not match batch fill.
Batch byte cap	FIXED MAX_BATCH_PAYLOAD=256 KiB. Count <= floor((262144 - 8)/(useful + 60)).	JSON sender_max_batch_bytes=1,048,416 is not enforced by the native splitter. Total pool-pull bytes=4,194,144 is enforced.	Pre-cloud gap: expose an enforced Delugi byte cap; review native Raptr byte-splitting behavior before sweeping it.
Batch cadence	CLI --batch-ms=10 (1..1000 ms). Flush on capacity or elapsed cadence.	JSON QS poll=150 ms; min nonempty=80 ms; max=300 ms; native dynamic backpressure also applies.	Different native batching policies. Record effective cut times and fill, not just a common label.
Proposal bytes / tx	Resource profile: 1 MiB or 32 MiB ordering wire budget. Value+witness allowance reserves mandatory HISTORY3 overhead.	JSON consensus.max_sending_block_txns=50,000; max_sending_block_bytes=30 MiB; inline=100 tx / 200 KiB.	Separate proposal budget from transport frame limit. Raptr message limit is separately 1 MiB by default.
Max payload batches	No independent max_payloads switch. A frontier checkpoint can cover many batches; MMR chunk capacity is not a batch-count cap.	JSON max_batches_per_pull=200; current native > checks can select 201. Native subblock count is fixed at 8.	Use explicit names: batch bytes, tx per batch, batches per proposal, proposal bytes, frame bytes.
Protocol bound Δ	CLI --bound-ms=250 default; acceptance used 100. PROG pacing defaults to Δ; native 7Δ / 3Δ catch-up rules remain.	JSON delta_ms; accepted runner fixes 100. Leader 4Δ; partial-QC wait 0.1Δ; fetch 2Δ; sync 15Δ; status 10Δ.	Δ is protocol configuration. It is not added network delay, nor a latency scoring deadline.
Network delay δ	External calibrated shaper. macOS helper 1..100 ms; Linux ingress netem delay 0..1000 ms and loss 0..100%.	No node delay knob. Current runner records 0 ms added delay; future deployment can reuse the external shaper.	Keep δ, Δ, batching and application hold distinct. Current helper schema has no bandwidth/jitter knobs.

Source index: D1-D7 and R1-R7 in docs/architecture/delugi-code-structure-2026-10-22.txt. All limits shown are current source facts.
The two Raptr cap qualifications are inherited native behavior, documented here without changing protocol implementation.

Faults, resources and the cloud boundary

n is committee size, f=(n-1)/3 is tolerated faults, and b is the number actually faulty. For the stated fault model require b <= f; n=4,7,31 gives f=1,2,10.

PARAMETER	DELUGI TODAY	RAPTR TODAY	EXPOSURE / EVALUATION MEANING
Committee n / f	CLI --nodes={4,7,31}; process validates 4..31 with n=3f+1. f is derived, not set independently.	JSON/keygen supports 4..31, n=3f+1. Accepted runner fixes n=4; node currently requires loopback addresses.	An n/f setting is not evidence that all n/f campaigns have passed. Current common-binary acceptance is n=4.
Actual faults b	Sustained runner: one paused replica OR one omission replica. Finite silent-first-leader fixture also exists.	Case presets: healthy (0); silent first leader (replica 1); missing-payload receiver 0 with first 3 QS messages dropped.	No general arbitrary-b Byzantine campaign in the common runners. Keep faulty IDs explicit and separate from f.
Fault timing / scope	CLI start/duration/peers; payload drops BATCH+DATA, all drops outgoing application protocol messages.	Preset pause, original-payload omission and application materialization hold; no general event schedule CLI.	Future schedule needs mode, IDs, targets, activation/withdrawal, opportunity witness and positive effect checks.
Storage completion	memory-visible-v1 default; optional shared-rocksdb-written-v1, fresh per-replica directory and typed cache.	Native QS uses shared RocksDB: WAL enabled, sync=false; completes at Written. Native quotas remain core-owned.	Match declared completion profile. Written is not fsync durability; Delugi core restart recovery is absent.
Queues / workers	CLI arrival-queue-capacity=65,536. Sync actor loop; TCP workers. Most transport limits are source/profile fixed.	JSON Limits exposes inbox/RPC/ingress/payload/task caps. Runtime fixed: Tokio 2, max blocking 32, Rayon 2.	Shared TCP default: 64 in-flight frames / 8 MiB per peer. Reused services do not imply equal worker budgets.
Observation / bounds	Full/compact receipts; compact omits detailed latency. --deadline-ms=1000 is scoring, not a protocol timeout.	Finite measurement=2000 ms + drain=2000 ms. JSON task limits=100k async / 1024 blocking; these are not RSS limits.	Primary latency: successful admission to verified contiguous output at the assigned correct origin, one monotonic clock.
Cloud launch	Legacy --inventory SSH/netem path exists. --shared-build-receipt currently rejects --inventory; disk acceptance local-only.	Common Raptr runner is localhost-only. Existing remote-native adapter covers other artifacts, not this Raptr port.	Common executable deployment still needs build/config/host-local start, full receipts, cleanup and resource reconciliation.

Before cloud parameter sweeps
Wire the common-node remote adapter; expose and verify the intended caps; add sustained Raptr ingress and bounded multi-fault schedules.

Source index: D8-D12, R8-R12, S1-S3. Proposed sweep schema must bind workload, batch, proposal, transport, timing, faults, storage and resources separately.
This document records current behavior and extension points. No cloud resources were provisioned and no protocol source was changed.